

## Orientações — TicketFlow: Sistema de Chamados Técnicos

### ✓ Observações

- Este projeto avalia **CSS Grid** como técnica principal de layout. As 3 páginas (`login.html`, `index.html`, `detalhe.html`) devem ter sua **estrutura geral** construída com `display: grid` — nunca com Flexbox ou float.
- **Flexbox só pode ser usado dentro de componentes** (alinhar ícone + texto, distribuir badges numa linha, alinhar avatar + nome, etc.), nunca para definir as colunas/áreas principais de uma página.
- **position só pode aparecer em um único lugar do projeto inteiro**: a navbar fixa (`position: fixed`) do `index.html` e do `detalhe.html`. Não use `position` na sidebar, em cards, em badges, em painéis ou em qualquer outro elemento. O `login.html` não tem navbar e, portanto, não usa `position` em lugar nenhum.
- Não é permitido nenhum JavaScript. A navegação entre as páginas é feita só com links (`<a>`) e com o `action` do formulário de login.
- O CSS deve ser dividido em **múltiplos arquivos**, separados por responsabilidade (variáveis, base, componentes, navbar, e um arquivo por página) — ver a seção 4 das boas práticas abaixo para a estrutura sugerida.
- Use **elementos semânticos de HTML5**: `header`, `nav`, `main`, `aside`, `section`, `article`, `footer`, `form`, `fieldset`, `legend`, `time`, etc.
- Não use IDs para estilização geral. Prefira **classes reutilizáveis** (`.card`, `.badge`, `.sidebar-link`, `.kpi-card`). IDs só para âncoras ou para associar `<label for="">` a um `<input id="">`.
- Todas as cores, espaçamentos e transições devem vir de **variáveis CSS** declaradas em `:root` — nenhum valor solto repetido pelo CSS.

### 🔍 Boas práticas adicionais

#### 1) Estrutura e organização do HTML

- Utilize indentação correta (2 ou 4 espaços) e consistente nas 3 páginas.
- Evite elementos extras desnecessários (`div` sem função quando um elemento semântico resolveria).
- Mantenha a hierarquia correta de títulos em cada página:
  - Cada página tem **apenas um** `<h1>`.
  - Títulos de seção/card usam `<h2>/<h3>` de forma consistente.
- Use `<p>` para textos corridos (descrição do chamado, frase de efeito do login), nunca `<div>`.
- Use `<ul>/<li>` para o histórico do chamado, links da sidebar e listas de diferenciais — nunca uma sequência de `<div>` soltos.
- Associe cada `<label>` ao seu campo com `for/id` nos formulários de login e de atualização de chamado.

#### 2) Organização e boas práticas no CSS

- Cada arquivo CSS tem **uma única responsabilidade** (ver a estrutura de arquivos na seção 4) — evite misturar, por exemplo, variáveis com estilos de componentes, ou o layout de uma página com o de outra.
- Dentro de cada arquivo, mantenha uma ordem lógica:
  - `variaveis.css`: só declarações em `:root` (cores, espaçamentos, raio de borda, duração de transição, fontes) — nenhum seletor de elemento aqui.
  - `base.css`: reset de `box-sizing: border-box`, margens e paddings, depois os elementos base (`body`, `h1-h3`, `p`, `a`, `ul`, `img`, campos de formulário).
  - `componentes.css`: os componentes reutilizáveis entre páginas (`.btn`, `.badge`, `.avatar`, estilo genérico de campo de formulário).
  - `navbar.css`: só os estilos da navbar.
  - `login.css`, `dashboard.css`, `detalhe.css`: o layout e os estilos exclusivos de cada página, com a media query daquela página no final do próprio arquivo — não em um arquivo separado.
- Antes de aplicar `display: grid` em qualquer container, identifique explicitamente **quantas colunas/linhas** ele deve ter e **quais elementos filhos diretos** vão ocupar cada área — escreva isso como comentário se ajudar. O erro mais comum é aplicar `grid-template-areas` no container errado (um ancestral distante em vez do pai direto dos itens).
- Nomeie as áreas do `grid-template-areas` com os mesmos nomes usados em `grid-area` nos elementos filhos — um nome errado faz o layout quebrar silenciosamente (o navegador não avisa).
- Para espaçamento, siga a escala definida na especificação do projeto: **4px / 8px / 16px / 24px / 32px / 48px**.
- Use nomes de classe em inglês, claros e descritivos:
  - Correto: `.ticket-card`, `.kpi-card`, `.sidebar-link`, `.status-badge`
  - Evitar: `.card1`, `.coisa`, `.div2`

### 3) Boas práticas visuais

- Mantenha a mesma paleta de cores nas 3 páginas — não invente cores novas.
- Todos os badges de status/prioridade devem usar sempre a mesma cor para o mesmo significado (ex.: vermelho é sempre "Aberto"/"Alta" em qualquer página).
- Todos os cards de chamado devem ter a mesma estrutura interna e proporções semelhantes.
- Garanta contraste legível entre texto e fundo, especialmente nos badges e na navbar (texto branco sobre índigo escuro).
- Botões seguem um padrão visual único: mesmo `border-radius`, mesma altura, mesmo peso de fonte — o botão primário (índigo) para ações principais, botões secundários (borda, sem preenchimento) para ações menos importantes.

### 4) Estrutura do projeto e arquivos

- Estrutura de pastas esperada:

```
seu-projeto/  
  login.html  
  index.html  
  detalhe.html  
  assets/  
    css/  
      variaveis.css  
      base.css  
      componentes.css  
      navbar.css  
      login.css  
      dashboard.css  
      detalhe.css
```

- Cada página importa, em `<link>`, **só os arquivos que ela realmente usa** — nessa ordem, já que os arquivos seguintes dependem dos anteriores (um componente usa variáveis, uma página usa componentes, etc.):
  - `login.html` → `variaveis.css`, `base.css`, `componentes.css`, `login.css` (sem `navbar.css` — essa página não tem navbar)
  - `index.html` → `variaveis.css`, `base.css`, `componentes.css`, `navbar.css`, `dashboard.css`
  - `detalhe.html` → `variaveis.css`, `base.css`, `componentes.css`, `navbar.css`, `detalhe.css`
- Nomeie os arquivos exatamente como acima (`login.html`, `index.html`, `detalhe.html`) para que os links entre as páginas funcionem sem ajustes.

### 5) Formulários e validação HTML5

- No login: `type="email"` no campo de e-mail, `required` e `minlength/pattern` no campo de senha. Teste enviando o formulário vazio e com uma senha curta — o navegador deve bloquear o envio e mostrar a mensagem nativa.
- No formulário de atualização do chamado: `<select>` e `<textarea>` marcados como `required`. Use o atributo `title` no campo com `pattern` para personalizar a mensagem de erro exibida pelo navegador.
- Não é necessário (nem permitido) validar nada via JavaScript — toda a validação é feita pelos atributos nativos do HTML5.

### 6) Acessibilidade mínima

- Todos os avatares (`<img>`) devem ter `alt` descritivo (ex.: `alt="Foto de Fernanda Lima"`).
- Links de navegação (sidebar, breadcrumb) devem ter texto compreensível — nunca "clique aqui".
- O botão **+** "Novo chamado" e os botões de ação rápida devem ter texto visível, não apenas ícone.
- Use `:focus-visible` para garantir contorno visível ao navegar pelos campos do formulário com o teclado (Tab).

### 7) Comentários úteis

Comentários curtos no HTML, marcando o início de cada bloco estrutural, ajudam bastante nesse projeto:

```
<!-- Navbar fixa -->  
<!-- Sidebar de navegação -->
```

```
<!-- Grid de chamados -->
```

Evite comentários que só descrevem o óbvio:

```
<!-- DIV 1 -->  
<!-- aqui começa o footer -->
```

---

## ❌ O que NÃO fazer

- Não usar Flexbox ou `float` para definir a estrutura geral de nenhuma das 3 páginas — isso é papel do Grid.
- Não usar `position` em nenhum elemento além da navbar fixa.
- Não usar nenhuma linha de JavaScript, nem para "simular" o login.
- Não usar tabelas (`<table>`) para layout.
- Não usar IDs para estilizar componentes repetidos (cards, badges).
- Não jogar tudo em um único arquivo CSS gigante — divida por responsabilidade, como descrito na seção 4.
- Não repetir a mesma regra em arquivos diferentes — se dois arquivos têm o mesmo seletor, essa regra provavelmente pertence a um arquivo mais genérico (`base.css` ou `componentes.css`).
- Não importar em uma página um arquivo CSS que ela não usa (ex.: `login.html` importando `navbar.css`).
- Não misturar valores de espaçamento fora da escala definida (4/8/16/24/32/48px).
- Não deixar cards, badges ou botões com aparência diferente entre as páginas (ex.: badge "Resolvido" com cores diferentes no dashboard e no detalhe).